Rheinische Friedrich-Wilhelms-Universität Bonn

# Bachelorstudiengang Informatik

# Prüfungsklausur

| | |
|---|---|
| Semester: WS ✚ SS □ *2015/16* | Versuch: 1. ✚ 2. □ |
| Modulbezeichnung: ***Systemnahe Programmierung*** | |
| Modulnummer: *BA-INF 034/129* | Prüfungsdatum: 23. *Februar 2015* |
| 1. Prüfer: *Prof. Dr. Matthew Smith* | 2. Prüfer: *Dr. Matthias Frank* |

vom Prüfling auszufüllen:

| | |
|---|---|
| Name: | Matrikel-Nr: |
| Vorname: | Semester: |
| **Studiengang**<br>Bachelor Informatik ☐ Lehramt ☐ Informatik als Nebenfach ☐ | |

vom Prüfer auszufüllen:

| |
|---|
| Bewertung: |

*Noten und Notenwerte: **sehr gut (1,0; 1,3) – gut (1,7; 2,0; 2,3) – befriedigend (2,7; 3,0; 3,3) – ausreichend (3,7; 4,0) – nicht ausreichend (5,0)**

Datum: 23. *Februar 2016*

---------- ----------

Unterschrift des 1. Prüfers Unterschrift des 2. Prüfers

Prof. Dr. Matthew Smith
Dr. Matthias Frank

Bonn, den 24. Februar 2015

**Hinweise zur**
**Modulprüfung zu „Systemnahe Programmierung“ (BA-INF 034/129)**
Wintersemester 2014/2015
9:30 Uhr – 11:00 Uhr

## Hinweise (GENAU DURCHLESEN!)

- Neben Papier und Schreibutensilien sind **keine weiteren Hilfsmittel** erlaubt. Verwenden Sie nur dokumentenechte Stifte (z.B. keine Bleistifte). Verwenden Sie keine roten Stifte.
- Vergessen Sie nicht, Ihren Namen und die Matrikelnummer auf *jedes* Blatt zu schreiben. Blätter ohne diese Angaben werden nicht gewertet.
- Schreiben Sie Ihre Lösungen auf die Aufgabenblätter möglichst in die dafür vorgesehenen Felder. Sie können auch die Rückseiten verwenden. Weiteres Schreibpapier kann von den Betreuern angefordert werden. Benutzen Sie kein mitgebrachtes Papier.
- Bitte schreiben Sie in Ihrem eigenen Interesse deutlich. Für nicht lesbare Lösungen können wir keine Punkte vergeben.
- Klausurblätter dürfen nicht voneinander getrennt werden.
- Werden mehrere unterschiedliche Lösungen für eine Aufgabe abgegeben, so wird die Aufgabe nicht gewertet.
- Im Fall von Täuschungsversuchen wird die Klausur sofort mit 0 Punkten bewertet. Eine Vorwarnung erfolgt nicht.
- In der Klausur können Sie 90 Punkte erhalten. Achten Sie darauf, dass Sie im Schnitt nicht viel mehr als 1 Minute pro Punkt zur Verfügung haben. Mit 50% dieser Gesamtpunktzahl haben Sie die Klausur sicher bestanden.

Viel Erfolg!

| **1** | **2** | **3** | **4** | **5** | **6** | **7** | **Gesamt** |
|---|---|---|---|---|---|---|---|
| Multiple Choice | IO-Mux | Assembler | Thread Sync | Go | Prozesse | Sockets TCP | |
| 12 | 12 | 15 | 20 | 14 | 5 | 12 | 90 |
| | | | | | | | |

**Aufgabe 1 :** (Multiple-Choice, 12 Punkte)

Bei den folgenden Multiple-Choice-Fragen wird für jedes richtig gesetzte Kreuz 1 Punkt vergeben. Bei falschen Antworten werden 0 Punkte vergeben.

| | Aussage | Richtig | Falsch |
|---|---|---|---|
| 1 | Bei der Kommunikation über Sockets braucht der Client die Funktion `bind()` nicht aufzurufen. | ☐ | ☐ |
| 2 | Ein neu generierter Socket ist per default im non-blocking Mode, kann aber per `fcntl()` in den blocking Mode gesetzt werden. | ☐ | ☐ |
| 3 | Bei Verwendung der Protokollfamilie AF_INET bestimmt der Sockettyp SOCK_STREAM oder SOCK_RAW, ob TCP oder UDP als Transportprotokoll verwendet wird. | ☐ | ☐ |
| 4 | Beim Preforked Server (mit TCP) erzeugt der Haupt-Prozess zunächst eine feste Anzahl Child-Prozesse zur parallelen Bearbeitung eingehender TCP-Verbindungen und läuft danach endlos weiter, damit die erzeugten Child-Prozesse nicht abgebrochen werden. | ☐ | ☐ |
| 5 | Ein Aufruf der Systemfunktion `read()` mit TCP liefert im Falle des Verbindungsabbaus von der Gegenseite den Rückgabewert 0 zurück. | ☐ | ☐ |
| 6 | Die sog. „Network-Byte-Order“ ist die eindeutige Darstellung von aus mehreren Bytes bestehenden Integer-Zahlen, die unabhängig von der konkreten Host-Architektur eingesetzt werden kann. | ☐ | ☐ |
| 7 | Das Prozessor-Status-Register kann mit Hilfe von bestimmten Sprungbefehlen zur Verzweigung in einem Assembler-Programm ausgewertet werden. | ☐ | ☐ |
| 8 | Threads laufen in getrennten Datenadressbereichen, teilen sich aber denselben Stack. | ☐ | ☐ |
| 9 | Unter Linux können Prozesse mit Hilfe von *FIFO*s über das Netzwerk kommunizieren. | ☐ | ☐ |
| 10 | Wenn ein Linux-Prozess mit geöffnetem File `fork()` aufruft, teilen sich der neue und der alte Prozess denselben File Offset. | ☐ | ☐ |
| 11 | In Go werden ALLE Variablen und Datenstrukturen direkt auf dem Stack gespeichert. | ☐ | ☐ |
| 12 | Die Verwendung von `select()` in Go ist immer blockend. | ☐ | ☐ |

Sollten Sie der Meinung sein, dass eine (oder mehrere) der obigen Fragen missverständlich mit „Richtig“ oder „Falsch“ zu beurteilen sind, dann können Sie unten Ihre gewählte Antwort **kurz** begründen. Bitte achten Sie dabei selbst auf Ihre Zeiteinteilung: Es macht keinen Sinn, viele (oder gar alle) der Multiple-Choice-Fragen durch ausführlichen Text zu begründen. Es gibt **nur** für die **oben** gewählten Antworten Punkte, nicht für Ihre Erläuterungen.

| Nr. | Ihre Erläuterung |
|---|---|
| | |
| | |
| | |
| | |
| | |

**Aufgabe 2:** (I/O-Multiplexing, 12 Punkte [4x1 + (4x1+ 2x2)])

a) Erläutern Sie kurz die Funktionen der Hilfsmakros zur Manipulation von fd-sets:

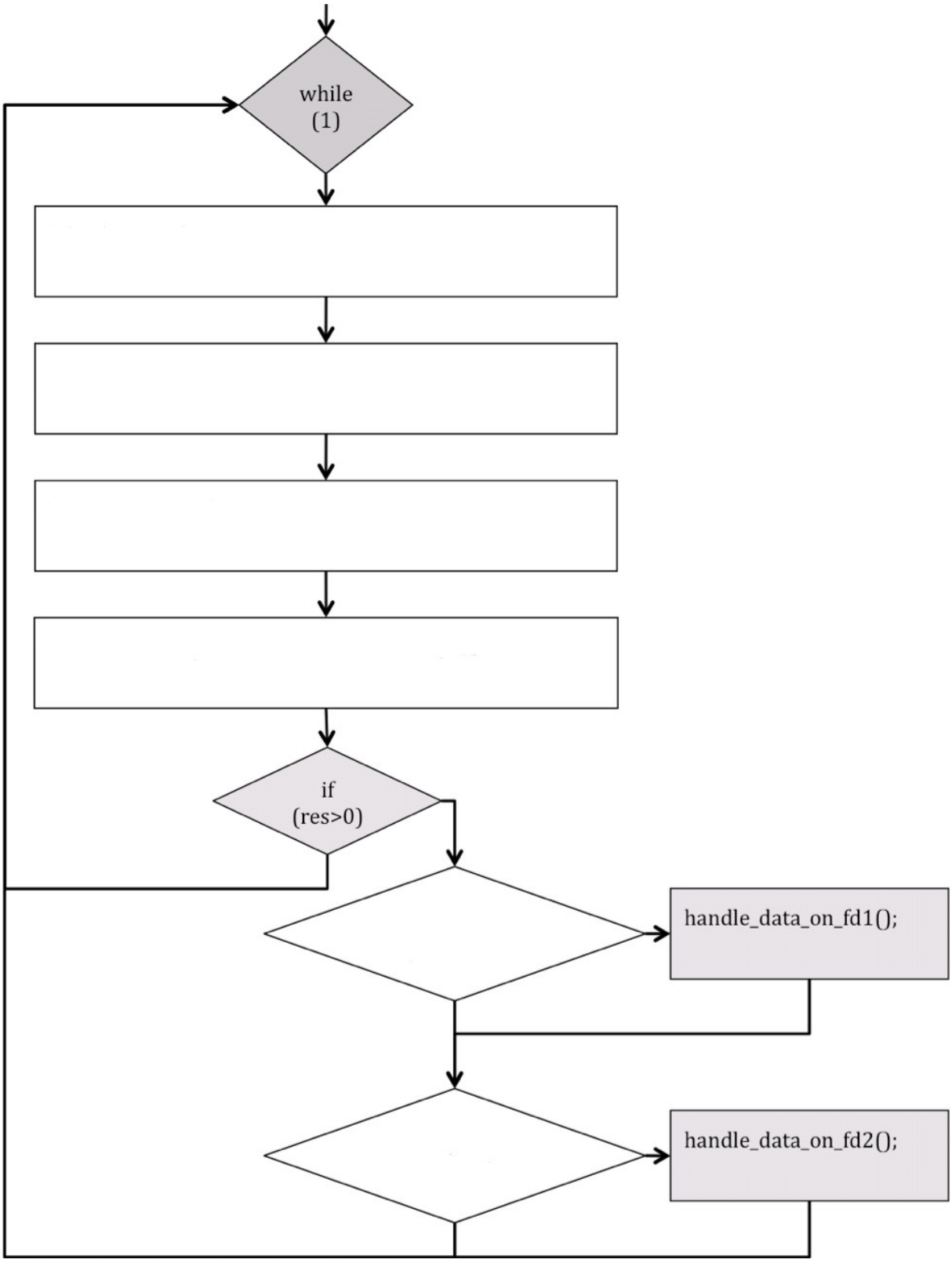
| FD_ZERO (fd_set *set) |
|---|

| FD_SET (int fd, fd_set *set) |
|---|

| FD_CLR (int fd, fd_set *set) |
|---|

| FD_ISSET (int fd, fd_set *set) |
|---|

b) Im folgendem ist ein Ablaufschema für select mit zwei File-Descriptoren, fd1 und fd2, sowie die Set fd_set gegeben. Auf fd1 und fd2 soll lesend zugegriffen werden und der select -Aufruf soll blockierend sein. Füllen Sie die Kästen mit validem C-Code, so dass select zum I/O-Multiplexing zwischen den beiden File Descriptoren genutzt wird.



**Aufgabe 3: (Assembler umschreiben, 15 Punkte [5 x 3 Punkte])**

In dieser Aufgabe sollen Sie 32-Bit x86-Assembler-Code (in AT&T-Syntax) in gleichbedeutenden C-Code umwandeln und umgekehrt (wahlweise in AT&T- oder Intel-Syntax).

Die Register EAX, EBX und ECX sollen dabei den 32-Bit Integer-Variablen a, b und c entsprechen. Außerdem wurde an der Speicheradresse des Labels feld ein int-Array der Länge 1024 erstellt, entsprechend der C-Deklaration int feld[1024].

Am Ende dieser Klausur finden Sie als Hilfestellung einen Auszug aus der x86-Befehlsrefenz. Beispiel: incl %eax entspricht a++;

a) Wandeln Sie folgende Assembler-Fragmente in äquivalente C-Befehlssequenzen um.

1.

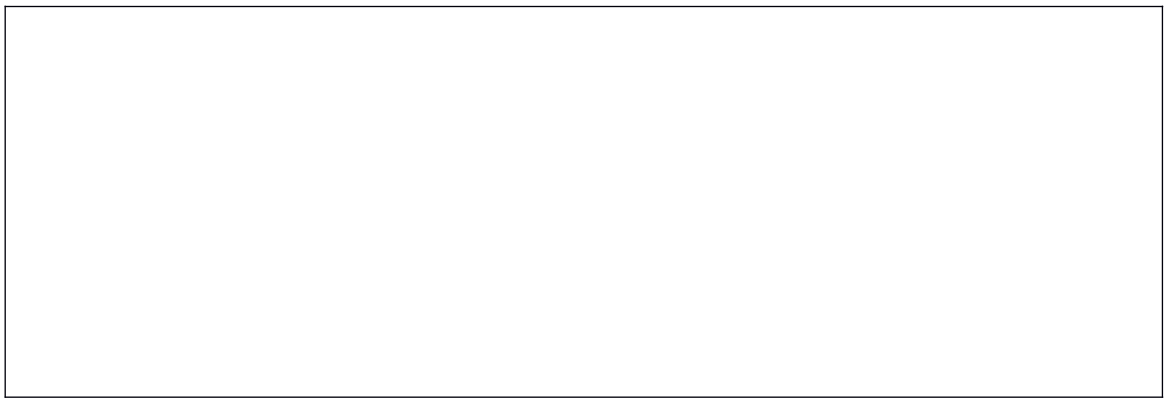
```
movl %eax, %ecx
subl %ebx, %ecx
```

2.

```
movl $7, %eax
imull %ebx, %eax
```

3.

```
     cmpl %ebx, %eax
     jge label
     incl %ecx
label:
```



b) Wandeln Sie die folgenden C-Instruktionen in x86-Assembler um, wahlweise in AT&T- oder Intel-Syntax.

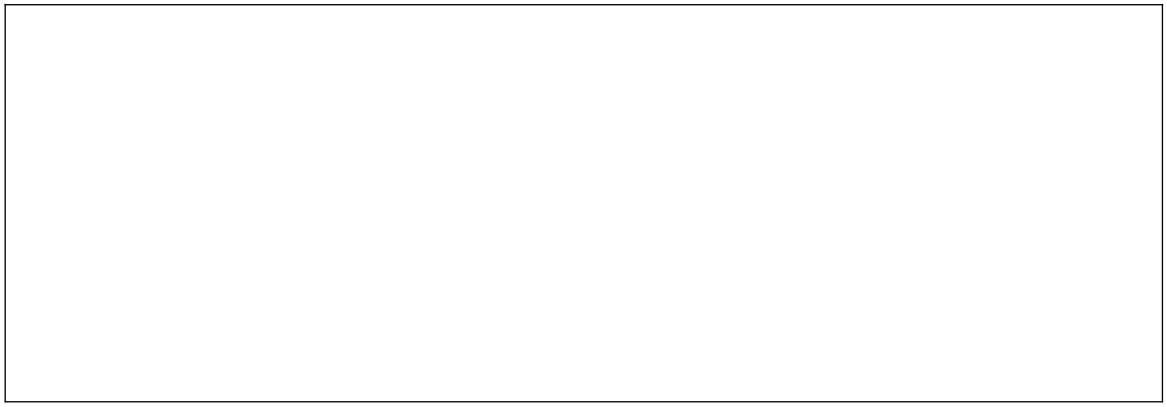
1.

```
c = (a+b) / 2;
```



2.

```
a = feld[5];
```



**Aufgabe 4:** (Thread-Synchronisation, insbes. Mutex, Semaphor, 20 Punkte + ggf. 2 Bonuspunkte [5 + 5 + 4 + 2 Bonus + 6])

a) (5 Punkte) In der Vorlesung wurden die folgenden fünf Konzepte für Thread-Synchronisation im Detail besprochen. Nennen Sie kurz und prägnant den Einsatzzweck bzw. charakteristische Eigenschaften des jeweiligen Konzeptes:

1. Mutex Variablen

_____

_____

2. Condition Variablen

_____

_____

3. Semaphore

_____

_____

4. Barriers

_____

_____

5. Read-write Locks

_____

_____

Betrachten Sie bei den weiteren Aufgabenteilen b) bis e) die folgenden gegebenen Funktionen, die allein mit Hilfe von Mutex-Variablen Semaphore realisieren - konkret die drei wesentlichen Funktionen für Semaphore, nämlich SEM_init(), SEM_wait() und SEM_post().
Auf eine Identifikation des Semaphors wird aus Gründen der besseren Lesbarkeit verzichtet, d.h. in einem Thread-fähigen Programm könnte nur eine Instanz unseres Semaphors benutzt werden.

```
int Sem_counter ;  // globale Counter-Variable

void SEM_init(int counter_init) {
     mutex_init(&CounterMutex, NULL)
     mutex_init(&SemMutex, NULL)

     mutex_lock(&CounterMutex);
     Sem_counter = counter_init;
     mutex_unlock(&CounterMutex);
}

void SEM_wait() {
     int block = 0;

     mutex_lock(&CounterMutex);
     Sem_counter--;
     printf("counter-- = %d\n", Sem_counter);
     If (Sem_counter <=0) block = 1;
     mutex_unlock(&CounterMutex);

     If (block) {
          printf("Vor lock = %d\n", Sem_counter);
          mutex_lock(&SemMutex);
          printf("Nach lock = %d\n", Sem_counter);
     }
}

void SEM_post() {
     mutex_lock(&CounterMutex);
     Sem_counter++;
     mutex_unlock(&CounterMutex);

     printf("post! = %d\n", Sem_counter);
     mutex_unlock(&SemMutex);
          // unlock eines nicht gelockten Mutex
          // bewirkt keinen Schaden
}
```

Die hier gegebene Implementation von SEM_wait() realisiert das Prinzip von wait() mit negativem Zähler:

1) Dekrementiere den Zähler um 1
2) Wenn Zähler < 0, dann blockiere
3) Nutze die Ressource (hier: verlasse SEM_wait() )

b) (5 Punkte) Skizzieren Sie die Ausgabe (siehe printf()-Statements in SEM_wait() und SEM_post() ) für folgenden zeitlichen Ablauf von Semaphor-Operationen. Nehmen Sie dabei an, dass die Funktionen SEM_wait() und SEM_post() nicht aufgrund von Thread-Scheduling unterbrochen werden (Ausnahme: Blockieren des Mutex <SemMutex> ist möglich).

SEM_init( 3)

SEM_wait()

SEM_wait()

SEM_wait()

SEM_wait()

SEM_wait()

SEM_post()

Schreiben Sie Ihre Lösung neben die Aufrufe oben oder hier unterhalb:

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

c) (4 Punkte) Die in der Aufgabe gegebene Implementation von SEM_wait() realisiert das Prinzip von wait() mit negativem Zähler:

1) Dekrementiere den Zähler um 1
2) Wenn Zähler < 0, dann blockiere
3) Nutze die Ressource (hier: verlasse SEM_wait() )

Der Vergleich auf den Zählerstand in SEM_wait() lautet jedoch:

```
If (Sem_counter <=0) block = 1;
```

Erläutern Sie, warum ein Vergleich auf „< 0“ nicht das gewünschte Ergebnis bringen würde! (Hinweis: bedenken Sie die printf()-Ausgaben aus Aufgabe b) auf dem Weg zu Ihrer Antwort)

d) **Zusatzfrage!!!** Durch beantworten dieser Frage können Sie bei dieser Aufgabe **2 zusätzliche Punkte** bekommen:

In welcher Hinsicht könnte die oben genannte Implementierung eines Semaphors durch Mutex-Variablen im realen Einsatz problematisch sein? Denken Sie an die quasi-parallele Ausführung von Threads und die mögliche Unterbrechung durch Thread-Scheduling.

e) (6 Punkte) Die gegebene Implementierung von SEM_wait() soll nun so verändert werden, dass das folgende Prinzip mit nicht-negativem Zähler für Semaphore realisiert wird:

1) Wenn Zähler = 0, dann blockiere
2) Dekrementiere den Zähler um 1
3) Nutze die Ressource (hier: verlasse SEM_wait() )

Schreiben Sie SEM_wait() nun entsprechend um. printf()-Ausgaben wie im gegebenen Beispiel brauchen dabei **nicht** enthalten sein. SEM_init() und SEM_post() brauchen nicht verändert werden (und sind zur Übersichtlichkeit nochmals dargestellt).

Denken Sie an korrekte Realisierung des Vergleiches des Zählerstandes (vgl. Aufgabenteil c).

```
int Sem_counter ;  // globale Counter-Variable

void SEM_init(int counter_init) {
     mutex_init(&CounterMutex, NULL)
     mutex_init(&SemMutex, NULL)

     mutex_lock(&CounterMutex);
     Sem_counter = counter_init;
     mutex_unlock(&CounterMutex);
}

void SEM_wait() {
```

**Platz für Ihre Lösung:**

```
} // Ende von SEM_wait()

void SEM_post() {
     mutex_lock(&CounterMutex);
     Sem_counter++;
     mutex_unlock(&CounterMutex);

     mutex_unlock(&SemMutex);
          // unlock eines nicht gelockten Mutex
          // bewirkt keinen Schaden
}
```

**Aufgabe 5:** (Synchronisation von Goroutinen, 14 Punkte [8+2+4])

Go erlaubt eine einfache Routinen-Synchronisation, welche hervorragend für das Producer/Consumer Paradigma geeignet ist.

a) (8 Punkte) Schreiben Sie ein Programm mit einer Producer Funktion welche eine url aus der Job-Queue ausliest und ihr Ergebnis mit der Consumer Funktion synchronisiert. Hierbei ist zu beachten, dass die Producer Methode die HTTP Anfragen macht, von welchen maximal 30 gleichzeitige Anfragen erlaubt werden sollen. Die ständig laufende Consumer-Routine speichert den Inhalt der Website (content Variable in der Consumer Funktion) in eine Datenbank und fügt alle Hyperlinks (urls Variable in Form eines Slices in der Consumer Funktion) wieder zur Job-Queue hinzu. Passen Sie hierbei auf, dass das Programm nicht terminiert.

(Lösung von Teil a) auf Seite 14, Reserve auf Seite 15)

(Teilaufgaben b) und c) auf Seite 16)

**Ihre Lösung:**

```
var consumerQueue = make(chan http.Response, 0)
var wg sync.WaitGroup

// Platz für Ihre Lösung:




func producer(_____________________________________________) {
      // Platz für Ihre Lösung:



      resp, _ := http.Get(url)

      // Platz für Ihre Lösung:




}
func consumer() {
      defer wg.Done()
      for resp := range consumerQueue {
            defer resp.Body.Close()
            body, _ := ioutil.ReadAll(resp.Body)
            urls, content, err := magic.MagicParse(body)
            db.save(content)

            // Platz für Ihre Lösung:



      }
}
func main() {
      wg.Add(1)
      go consumer()

      // Platz für Ihre Lösung:




}
```

**Reserve-Platz für Ihre Lösung:**
(Bitte streichen Sie eine ungültige Lösung deutlich)

```
var consumerQueue = make(chan http.Response, 0)
var wg sync.WaitGroup

// Platz für Ihre Lösung:




func producer(_____________________________________________) {
      // Platz für Ihre Lösung:




      resp, _ := http.Get(url)

      // Platz für Ihre Lösung:




}
func consumer() {
      defer wg.Done()
      for resp := range consumerQueue {
            defer resp.Body.Close()
            body, _ := ioutil.ReadAll(resp.Body)
            urls, content, err := magic.MagicParse(body)
            db.save(content)

            // Platz für Ihre Lösung:




      }
}
func main() {
      wg.Add(1)
      go consumer()

      // Platz für Ihre Lösung:




}
```

b) (2 Punkte) Was macht das defer Statement in der ersten Zeile der Consumer Funktion und was hat es für einen Vorteil sowas zu verwenden?

c) (4 Punkte) Durch das Verwenden von http Methoden in der Consumer Funktion wird unter Umständen bei jeder Ausführung der Funktion neuer Speicher allokiert und beim Debuggen merken Sie, dass durch die Nebenläufigkeit der vielen Jobs viel Rechenzeit für den Garbage-Collector verschwendet wird und dass Sie zusätzlich noch eine sehr hohe Arbeitsspeicherauslastung haben. Beschreiben Sie das Problem und ein Konzept aus der Vorlesung um die Speicher- und Garbage-Collector-Auslastung zu reduzieren.

**Aufgabe 6: :** (Prozesse, 5 Punkte)

Bei der Ausführung des folgenden einfachen Programms treten nach kurzer Zeit Fehler auf und es wird immer wieder „Fork Failed“ auf dem Bildschirm ausgegeben.

**Hinweis:** Am Ende der Klausur finden Sie eine Liste mit der Syntax relevanter Bibliotheksfunktionen.

```
 1:  #include <unistd.h>
 2:  #include <stdio.h>
 3:  #include <stdlib.h>
 4:
 5:  void do_child_action() {
 6:       fprintf(stdout, "Child Action %d\n", getpid());
 7:       sleep(1);
 8:  }
 9:
10:  void handle_request() {
11:       pid_t  pid;
12:       pid = fork();
13:       if (pid < 0) {
14:            fprintf(stderr, "Fork Failed\n");
15:       } else if (pid == 0) {
16:            do_child_action();
17:            exit(0);
18:       }
19:  }
20:
21:  int main(int argc, char* argv[]) {
22:       while (1) {
23:            handle_request();
24:       }
25:  }
```

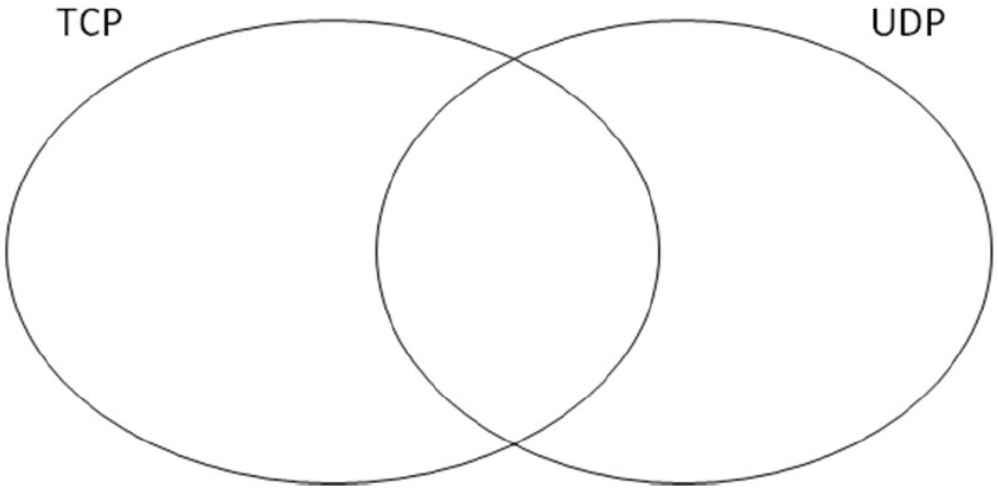
Erklären Sie in kurzen Worten, was der Programmierer hier falsch gemacht hat. Welche Schritte sind notwendig, um das Problem zu beheben?

(bei Bedarf noch Platz für Aufgabe 6)

(Erklären Sie in kurzen Worten, was der Programmierer hier falsch gemacht hat. Welche Schritte sind notwendig, um das Problem zu beheben?)

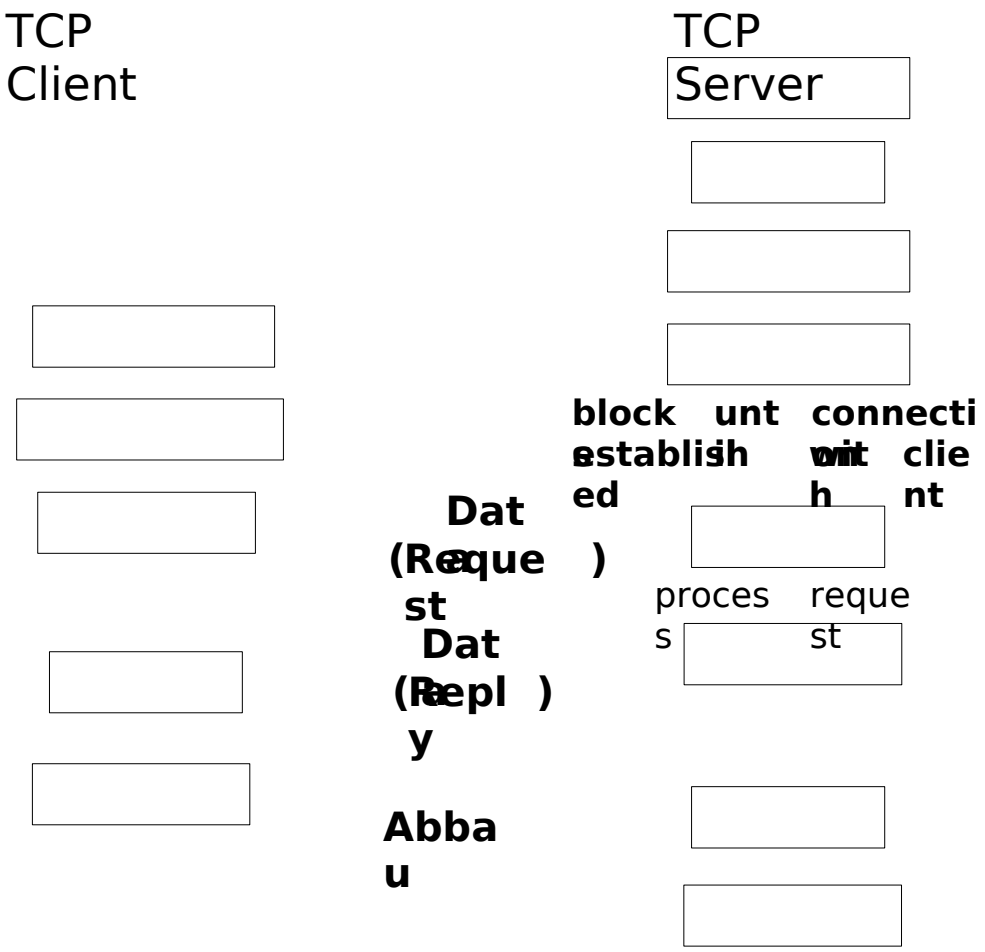
**Aufgabe 7:** (Serverstrukturen in C, 12 Punkte [11x0,5+13x0,5])

a) Ordnen Sie die folgenden Funktionsnamen zum Umgang mit Sockets den Protokollen UDP und TCP zu, indem Sie sie in den jeweiligen Mengenkreis eintragen. Ordnen Sie die Funktionen, die für beide Protokolle Verwendung finden, dem Schnittbereich der beiden Mengen zu.




Die zuzuordnenden Funktionsnamen sind: accept(), bind(), close(), connect(), listen(), read(), recvfrom(), sendto(), shutdown(), socket(), write().

b) Ergänzen Sie das angegebene allgemeine Ablaufschema einer UDP Client/Server Kommunikation über Sockets mit den entsprechenden C-Funktionsaufrufen. Vernachlässigen Sie dabei die Parameter der Funktionen.



## Signaturen relevanter Bibliotheksfunktionen

| |
|---|
| int **pthread_create**(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg); |
| int **pthread_attr_init**(pthread_attr_t *attr); |
| int **pthread_attr_destroy**(pthread_attr_t *attr); |
| void **pthread_exit**(void *value_ptr); |
| int **pthread_join**(pthread_t thread, void **value_ptr); |
| int **pthread_detach**(pthread_t thread); |
| int **pthread_cancel**(pthread_t thread); |
| int **pthread_setcancelstate**(int state, int *oldstate); |
| int **pthread_setcanceltype**(int type, int *oldtype); |
| void **pthread_testcancel**(void); |
| int **pthread_mutex_init**(pthread_mutex_t *mutex, pthread_mutexattr_t *attr); |
| int **pthread_mutex_destroy**(pthread_mutex_t *mutex); |
| int **pthread_mutexattr_init**(pthread_mutexattr_t *attr); |
| int **pthread_mutexattr_destroy**(pthread_mutexattr_t *attr); |
| int **pthread_mutexattr_settype**(pthread_mutexattr_t *attr, int type); |
| int **pthread_mutexattr_gettype**(const pthread_mutexattr_t *attr, int *type); |
| int **pthread_mutex_lock**(pthread_mutex_t *mutex); |
| int **pthread_mutex_trylock**(pthread_mutex_t *mutex); |
| int **pthread_mutex_unlock**(pthread_mutex_t *mutex); |
| int **pthread_cond_init**(pthread_cond_t *cond, const pthread_condattr_t *attr); |
| int **pthread_cond_destroy**(pthread_cond_t *cond); |
| int **pthread_condattr_init**(pthread_condattr_t *attr); |
| int **pthread_condattr_destroy**(pthread_condattr_t *attr); |
| int **pthread_cond_wait**(pthread_cond_t *cond, pthread_mutex_t *mutex); |
| int **pthread_cond_timedwait**(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime); |
| int **pthread_cond_signal**(pthread_cond_t *cond); |
| int **pthread_cond_broadcast**(pthread_cond_t *cond); |
| int **sem_init**(sem_t *sem, int pshared, unsigned int value); |
| int **sem_destroy**(sem_t *sem); |
| int **sem_wait**(sem_t *sem); |
| int **sem_trywait**(sem_t *sem); |
| int **sem_post**(sem_t *sem); |
| sem_t ***sem_open**(const char *name, int oflag); |
| sem_t ***sem_open**(const char *name, int oflag, mode_t mode, unsigned int value); |
| int **sem_close**(sem_t *sem); |
| int **sem_unlink**(const char *name); |
| int **pthread_barrier_init**(pthread_barrier_t *restrict barrier, const pthread_barrierattr_t *restrict attr, unsigned count); |
| int **pthread_barrier_destroy**(pthread_barrier_t *barrier); |
| int **pthread_barrier_wait**(pthread_barrier_t *); |
| int **pthread_rwlock_init**(pthread_rwlock_t *rwlock, const pthread_rwlockattr_t *attr); |
| int **pthread_rwlock_destroy**(pthread_rwlock_t *rwlock); |
| int **pthread_rwlock_rdlock**(pthread_rwlock_t *rwlock); |
| int **pthread_rwlock_tryrdlock**(pthread_rwlock_t *rwlock); |
| int **pthread_rwlock_trywrlock**(pthread_rwlock_t *rwlock); |
| int **pthread_rwlock_unlock**(pthread_rwlock_t *rwlock); |
| pid_t **getpid**(void); |

| |
|---|
| pid_t **getppid**(void); |
| int **system**(const char *command); |
| pid_t **fork**(void); |
| int **execl**(const char *path, const char *arg0, ... /*, (char *)0 */); |
| int **execv**(const char *path, char *const argv[]); |
| int **execle**(const char *path, const char *arg0, ... /*, (char *)0, char *const envp[]*/); |
| int **execve**(const char *path, char *const argv[], char *const envp[]); |
| int **execlp**(const char *file, const char *arg0, ... /*, (char *)0 */); |
| int **execvp**(const char *file, char *const argv[]); |
| void **exit**(int status); |
| void **abort**(void); |
| pid_t **wait**(int *stat_loc); |
| pid_t **waitpid**(pid_t pid, int *stat_loc, int options); |
| int **pipe**(int fd[2]); |
| int **dup**(int fd); |
| int **dup2**(int oldfd, int newfd); |
| FILE ***popen**(const char *cmdstring, const char *type); |
| int **pclose**(FILE *fp); |
| int **mkfifo**(const char *pathname, mode_t mode); |
| mqd_t **mq_open**(const char *name, int oflag); |
| mqd_t **mq_open**(const char *name, int oflag, mode_t mode, struct mq_attr *attr); |
| mqd_t **mq_getattr**(mqd_t mqdes, struct mq_attr *attr); |
| mqd_t **mq_setattr**(mqd_t mqdes, struct mq_attr *newattr, struct mq_attr *oldattr); |
| int **mq_send**(mqd_t mqdes, const char *msg_ptr, size_t msg_len, unsigned int msg_prio); |
| ssize_t **mq_receive**(mqd_t mqdes, char *msg_ptr, size_t msg_len, unsigned int *msg_prio); |
| int **mq_close**(mqd_t mqdes); |
| int **mq_unlink**(const char *name); |
| int **mq_notify**(mqd_t mqdes, const struct sigevent *notification); |
| int **kill**(pid_t pid, int signo); |
| int **raise**(int signo); |
| int **sigaction**(int signo, const struct sigaction *act, struct sigaction *oact); |
| int **sigprocmask**(int how, const sigset_t *set, sigset_t *oset); |
| int **pthread_sigmask**(int how, const sigset_t *set, sigset_t *oset); |
| int **sigpending**(sigset_t *set); |
| int **sigemptyset**(sigset_t *set); |
| int **sigfillset**(sigset_t *set); |
| int **sigaddset**(sigset_t *set, int signo); |
| int **sigdelset**(sigset_t *set, int signo); |
| int **sigismember**(const sigset_t *set, int signo); |
| int **pause**(void); |
| int **sigsuspend**(const sigset_t *sigmask); |
| int **sigwait**(const sigset_t *set, int *sig); |
| ssize_t **read**(int filedes, void *buf, size_t nbytes); |
| ssize_t **write**(int fildes, const void *buf, size_t nbyte); |
| int **close**(int filedes); |
| int **fcntl**(int fildes, int cmd, ...); |
| off_t **lseek**(int filedes, off_t offset, int whence); |
| int **ioctl**(int fildes, int request, ... /* arg */); |
| int **socket**(int *domain*, int *type*, int *protocol*); |
| int **bind**(int *sockfd*, const struct sockaddr **my_addr*, socklen_t *addrlen*); |

| |
|---|
| |
| uint16_t **htons**(uint16_t hvalue); |
| uint32_t **htonl**(uint32_t hvalue); |
| uint16_t **ntohs**(uint16_t hvalue); |
| uint32_t **ntohl**(uint32_t hvalue); |
| int **sendto**(int sockfd, const void *buff, size_t nbytes, int flags, const struct sockaddr *to, socklen_t tolen); |
| int **select**(int maxfdp1, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout); |
| int **recvfrom**(int sockfd, void *buff, size_t nbytes, int flags, struct sockaddr *from, socklen_t *fromlen); |
| in_addr_t **inet_addr**(const char *dotted); |
| char ***inet_ntoa**(struct in_addr network); |
| int **listen**(int sockfd, int backlog); |
| int **connect**(int sockfd, const struct sockaddr *serv_addr,socklen_t addrlen); |
| int **accept**(int sockfd, struct sockaddr *addr,socklen_t *addrlen); |
| int **close**(int sockfd); |
| int **shutdown**(int sockfd, int howto); |
| **FD_ZERO** (fd_set *set); |
| **FD_SET** (int fd, fd_set *set); |
| **FD_CLR** (int fd, fd_set *set); |
| **FD_ISSET**(int fd, fd_set *set); |
| int **getsockname**(int socket, struct sockaddr *restrict address, socklen_t *restrict address_len); |